

PulsePoll Backend API Specification

Reference for implementing the backend: REST endpoints for templates and poll lifecycle, and the SignalR hub contract for real-time join, answer and close. Based on the SRE/SignalR guide (auth model: none, poll-code + operatorToken only).

1. Data models

Template

Field	Type	Notes
id	string (UUID)	
operatorToken	string	Soft ownership only, not a security boundary
title	string	Required
showLiveResults	boolean	Per-template setting; controls whether participants see live tallies
sourceFilename	string null	Set only if created via upload
createdAt	ISO 8601 datetime	
questions	Question[]	1 or more

Question

Field	Type	Notes
text	string	Required, non-empty
options	string[]	2–4 entries, required
correctOptionIndex	int null	Optional. 0-based index into options. null/absent = opinion question, no correct answer. Present only when the operator explicitly marked one option correct (quiz/classroom use)

Poll (Temporary — fast cache, not the durable store)

Field	Type	Notes
pollCode	string	5–6 chars, alphanumeric, excludes 0/O, 1/I/l
templateId	string	Snapshot of template at start time — later template edits never affect a running poll
operatorToken	string	

Field	Type	Notes
status	"Open" "Closed"	
currentQuestionIndex	int	
tally	{ [questionIndex]: number[] }	Vote counts per option, per question
answeredBy	{ [questionIndex]: Set<connectionId> }	Dedup — keyed on connection, not identity
createdAt / closedAt	ISO 8601	Drives TTL

2. REST API

Base path: /api

2.1 POST /templates

Create a template directly from a structured payload (in-browser builder path — no parsing step).

Request body

```
{
  "operatorToken": "op_9f2a...",
  "title": "Team retro pulse check",
  "showLiveResults": true,
  "questions": [
    { "text": "What went well this sprint?",
      "options": ["Planning","Collaboration","Delivery speed","Nothing much"],
      "correctOptionIndex": null }
  ]
}
```

Response 201

```
{ "id": "tmpl_123", "title": "...", "questionCount": 1, "createdAt": "..." }
```

Validation errors 400: missing title; a question with < 2 options or > 4 options; correctOptionIndex out of range for its question's options.

2.2 POST /templates/upload

Accept a raw .csv or .json file, parse and validate it, and return a preview — nothing is persisted yet.

Request: multipart/form-data, field file. Size-capped at 1 MB. Extension checked before parsing.

CSV schema (header row required):

```
question,option1,option2,option3,option4,correctOption
"What's your favorite color?","Red","Blue","Green","Yellow",
"Is the sky blue?","Yes","No",,,1
```

correctOption is 1-based, optional, blank = no correct answer.

JSON schema:

```
{
  "title": "Team retro pulse check",
  "showLiveResults": true,
  "questions": [
```

```
{ "text": "What's your favorite color?", "options": ["Red","Blue","Green","Yellow"] },
{ "text": "Is the sky blue?", "options": ["Yes","No"], "correctOption": 1 }
]
```

Response 200 (preview, not yet saved)

```
{
  "previewToken": "prev_abc123",
  "title": "Team retro pulse check",
  "showLiveResults": true,
  "questions": [
    { "text": "What's your favorite color?", "options": ["Red","Blue","Green","Yellow"],
      "correctOptionIndex": null },
    { "text": "Is the sky blue?", "options": ["Yes","No"], "correctOptionIndex": 0 }
  ]
}
```

Response 422 on the first structural error — reject the whole file, never silently drop a row:

```
{ "error": "invalid_row", "row": 4, "reason": "missing required field 'question'" }
```

Other rejection reasons: `too_few_options`, `too_many_options`, `correct_option_out_of_range`, `correct_option_not_numeric`, `unsupported_extension`, `file_too_large`.

2.3 POST /templates/confirm

Persist a previously returned preview once the operator confirms it in the UI.

```
{ "previewToken": "prev_abc123", "operatorToken": "op_9f2a..." }
```

Response 201: same shape as POST /templates. Response 410 if the preview token expired or was already confirmed/discarded.

POST /templates/discard: { "previewToken": "..." } → 204. Raw upload is not retained after discard or after 30 days, whichever is first.

2.4 GET /templates?operatorToken=

List saved templates for this operator (metadata only, no full question bodies — keeps payload light for the template list screen).

```
[
  { "id": "tmpl_123", "title": "Team retro pulse check", "questionCount": 3, "createdAt":
    "...", "sourceFilename": null }
]
```

2.5 GET /templates/{id}

Full template including questions and correctOptionIndex per question (used to render the operator's preview-before-start screen). Requires matching operatorToken.

2.6 PUT /templates/{id} / DELETE /templates/{id}

Edit or delete a template. 404 if operatorToken doesn't match (soft ownership — treat as “not found,” not “forbidden,” since there's no real auth to enforce a distinct 403).

2.7 POST /polls

Start a poll from a saved template. Plain HTTP, not a hub method — needs a durable-adjacent write and a generated code before any realtime connection is useful.

```
{ "templateId": "tmpl_123" }
```

Response 201

```
{ "pollCode": "7F3KQ", "operatorToken": "op_9f2a...", "questionCount": 3 }
```

Poll state is copied from the template into the fast cache at this point; the template is not referenced live again.

2.8 GET /polls/{pollCode}/export

Operator-only (operatorToken query param or header), available while the poll is open or during its post-close retention window. Returns CSV: one row per question, per-option vote counts, and a correctOption column populated only for questions that have one set.

3. SignalR hub — PollHub

Client → server methods

Method	Params	Behavior
JoinPoll	pollCode	Validates code + poll open; adds connection to the pollCode group; returns PollJoined or JoinRejected. Rate-limited per connection and per source IP — primary defense against code enumeration.
SubmitAnswer	pollCode, questionIndex, optionIndex	Rejects if poll closed, question index stale, option invalid, or this connectionId already answered this question. Otherwise increments tally, broadcasts AnswerTally to the group (coalesce broadcasts every 250–500ms once group size > a few hundred), returns AnswerAccepted. Rate-limited per connection.
NextQuestion	pollCode	Operator only — validate operatorToken on the connection. Advances currentQuestionIndex, broadcasts QuestionChanged with the new question and a zeroed tally.
ClosePoll	pollCode	Operator only. Sets status Closed, freezes tally, broadcasts PollClosed(finalTally) to the whole group so anyone mid-submission gets this instead of a silent timeout.
GetPollSnapshot	pollCode	For reconnecting clients — SignalR gives a new connectionId on reconnect and does not replay group membership, so the client must explicitly re-join and re-fetch state. Returns the same shape as PollJoined.

Server → client events

Event	Payload	Sent to
PollJoined	{ question, options, questionIndex, questionCount, currentTally, showLiveResults }	Joining connection
JoinRejected	{ reason: "invalid_code" "poll_closed" "rate_limited" }	Joining connection
AnswerAccepted	{ questionIndex }	Answering connection
AnswerRejected	{ reason: "poll_closed" "duplicate" "invalid_option" "stale_question" }	Answering connection
AnswerTally	{ questionIndex, counts: number[] }	Whole group — only if showLiveResults is true for the poll's template; otherwise suppressed for participant connections (operator's own connection always receives it)
QuestionChanged	{ question, options, questionIndex, questionCount }	Whole group
PollClosed	{ finalTally: number[][], perQuestionCorrect: (number null)[] }	Whole group. perQuestionCorrect[i] is null for questions with no correct answer set — the client shows the “correct answer” badge only where this is non-null.

Reconnection contract

On onreconnected, the client must call JoinPoll again (new connectionId) and GetPollSnapshot, never assume prior group membership survived. This is the single most common cause of “stopped seeing updates” reports per the SRE guide — implement it as a non-optional step in the client SDK wrapper, not left to each screen.

4. Validation & rejection rules

- Every SubmitAnswer option is checked against the poll's actual stored option set — never trust a client-supplied index or tally delta.
- Duplicate-vote prevention is keyed on connectionId. A reconnect issues a new connectionId, so exactly-once-per-human voting is explicitly out of scope without added client-side state or auth — document this as a known limitation, not a bug.
- correctOptionIndex is validated at template save/parse time ($0 \leq \text{index} < \text{options.length}$) and never re-validated at answer time — it plays no role in accepting or rejecting a vote, only in the results view.

5. Rate limiting & security

Target	Limit basis	Purpose
JoinPoll	per connection + per source IP	Poll-code brute-force / scanning defense

Target	Limit basis	Purpose
SubmitAnswer	per connection	Prevent a single scripted client flooding tallies
POST /templates/upload	per operatorToken / IP	Abuse prevention on parsing endpoint

- TLS only (wss:// / https://); poll codes and answers otherwise travel in the clear.
- No PII collected anywhere — don't add fields for name/email “just in case.”
- Poll codes are released back to the available pool only after a delay past purge, to avoid immediate reuse confusion.

6. Data lifecycle / TTL

Data	Store	Retention
Template + metadata	Durable store	Indefinite, until operator deletes
Uploaded raw file	Blob storage	30 days, then purged
Poll definition, tally, dedup map, poll code	Fast cache	Lifetime of poll + short grace period after close (covers GET /polls/{code}/export and late joiners)

7. HTTP status code summary

Code	Meaning here
200	Successful read / preview
201	Resource created (template, poll)
204	Successful delete / discard
400	Malformed request body
404	Not found, or operatorToken mismatch (soft ownership)
410	Expired preview token
422	Structural validation failure on upload (row-identified)
429	Rate limited (JoinPoll, SubmitAnswer, upload)